AI Assisted Coding

AI for Data Processing: Data Cleaning and Preprocessing Scripts

NAME: K. SAI VARDHAN REDDY

ROLL NO: 2503A51266

ASSIGNMENT NO: 17

Task 1 – Employee Dataset Cleaning

AI Prompt

Act as a Python data-cleaning assistant. Create an employee dataset and clean it by filling missing salary and experience values with medians, filling missing designation with the mode, removing duplicate employee IDs, converting job titles and employment types to lowercase, and encoding categorical columns. Display the cleaned dataset and remaining missing-value counts.

Code Screenshot (VS Code-style editor)

Visual Studio Code | lab_assignment_17.py

Task 1 – Employee Dataset Cleaning • Python

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Sample employee data (the assignment does not provide a dataset)
df = pd.DataFrame({
    "employee_id": [101, 102, 103, 103, 104, 105],
    "salary": [45000, None, 52000, 52000, 60000, None],
    "designation": ["Manager", "Developer", None, None, "HR", "Developer"],
    "experience": [5, 2, None, None, 8, 3],
    "employment_type": ["Full-Time", "Part-Time", "Full-Time",
                        "Full-Time", "Contract", "Part-Time"]
})

# Fill numerical missing values with median
df["salary"] = df["salary"].fillna(df["salary"].median())
df["experience"] = df["experience"].fillna(df["experience"].median())

# Fill missing designation with the most frequent designation
df["designation"] = df["designation"].fillna(df["designation"].mode()[0])

# Remove duplicate employee IDs and standardize job titles
df = df.drop_duplicates(subset="employee_id", keep="first")
df["designation"] = df["designation"].str.lower().str.strip()
df["employment_type"] = df["employment_type"].str.lower().str.strip()

# Encode categorical columns
for col in ["designation", "employment_type"]:
    df[col + "_encoded"] = LabelEncoder().fit_transform(df[col])

print("Clean Employee Dataset:")
print(df.to_string(index=False))
print("\\nMissing values after cleaning:")
print(df.isnull().sum())
```

Figure: Python code for Task 1 – Employee Dataset Cleaning.

Code Explanation

1. Median is used for missing salary and experience because it is less affected by extreme values.
2. The most frequent designation (mode) fills missing job titles.
3. Duplicate employee IDs are removed, and text is converted to lowercase with extra spaces removed.
4. LabelEncoder converts designation and employment_type into numeric labels for analysis.

Expected Output

```
Clean Employee Dataset:
 employee_id  salary designation  experience employment_type  designation_encoded
employment_type_encoded
         101 45000.0     manager         5.0       full-time                    2
1
         102 52000.0   developer         2.0       part-time                    1
2
         103 52000.0   developer         3.0       full-time                    1
1
         104 60000.0          hr         8.0        contract                    0
0
         105 52000.0   developer         3.0       part-time                    1
2

Missing values after cleaning:
employee_id                 0
salary                      0
designation                 0
experience                  0
employment_type             0
designation_encoded         0
employment_type_encoded     0
```

Task 2 – Student Performance Data Preprocessing

AI Prompt

Act as a Python data-preprocessing assistant. Create student performance data, convert exam_date to datetime, extract month, day and semester, detect outliers in total_marks using the IQR method, handle them by clipping to IQR limits, normalize marks to the 0–1 range, and display the transformed dataset.

Code Screenshot (VS Code-style editor)

Visual Studio Code | lab_assignment_17.py

Task 2 – Student Performance Data Preprocessing • Python

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Sample student marks data
df = pd.DataFrame({
    "student_id": [1, 2, 3, 4, 5, 6],
    "exam_date": ["2026-01-12", "2026-02-15", "2026-03-10",
                  "2026-04-18", "2026-05-20", "2026-06-11"],
    "total_marks": [72, 85, 90, 78, 88, 250]
})

# Convert exam_date to datetime and extract time features
df["exam_date"] = pd.to_datetime(df["exam_date"])
df["month"] = df["exam_date"].dt.month
df["day"] = df["exam_date"].dt.day
df["semester"] = df["month"].apply(
    lambda m: "Semester 1" if m <= 6 else "Semester 2"
)

# Detect outliers using IQR and replace them with upper/lower limits
q1 = df["total_marks"].quantile(0.25)
q3 = df["total_marks"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
df["total_marks"] = df["total_marks"].clip(lower, upper)

# Normalize marks to a 0–1 range
scaler = MinMaxScaler()
df["normalized_marks"] = scaler.fit_transform(df[["total_marks"]])

print("Preprocessed Student Performance Data:")
print(df.to_string(index=False))
print("\\nIQR limits:", round(lower, 2), "to", round(upper, 2))
```

Figure: Python code for Task 2 – Student Performance Data Preprocessing.

Code Explanation

5. The exam_date column is converted into pandas datetime format.
6. Month, day and semester are extracted as additional academic features.
7. The IQR method identifies unusually high or low marks; clipping limits extreme values to the calculated boundaries.
8. MinMaxScaler converts marks to a range from 0 to 1.

Expected Output

```
Preprocessed Student Performance Data:
 student_id  exam_date  total_marks  month  day   semester  normalized_marks
          1 2026-01-12         72.0      1   12 Semester 1          0.000000
          2 2026-02-15         85.0      2   15 Semester 1          0.073034
          3 2026-03-10         90.0      3   10 Semester 1          0.101124
          4 2026-04-18         78.0      4   18 Semester 1          0.033708
          5 2026-05-20         88.0      5   20 Semester 1          0.089888
          6 2026-06-11        102.5      6   11 Semester 1          1.000000

IQR limits: 53.5 to 102.5
```

Task 3 – Student Health Data Preparation

AI Prompt

Act as a Python data-preprocessing assistant. Create sample student health records, fill missing height, weight and BMI with medians, encode medical_condition and fitness_status, standard-scale numerical features, split the data into training and testing sets, and display the prepared data and split sizes.

Code Screenshot (VS Code-style editor)

Visual Studio Code | lab_assignment_17.py

Task 3 – Student Health Data Preparation • Python

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

# Sample health records (illustrative only)
df = pd.DataFrame({
    "student_id": range(1, 9),
    "height": [165, 170, None, 160, 175, 168, 172, 158],
    "weight": [60, None, 70, 55, 78, 65, 72, 52],
    "BMI": [22.0, 23.5, None, 21.5, 25.5, 23.0, 24.3, 20.8],
    "medical_condition": ["None", "Asthma", "None", "None",
                          "Allergy", "Asthma", "None", "Allergy"],
    "fitness_status": ["Good", "Average", "Good", "Good",
                       "Average", "Good", "Average", "Good"]
})

# Fill missing numerical values with column medians
numeric_cols = ["height", "weight", "BMI"]
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())

# Encode categorical features
for col in ["medical_condition", "fitness_status"]:
    df[col + "_encoded"] = LabelEncoder().fit_transform(df[col])

# Standardize numerical features
scaler = StandardScaler()
df[["height_scaled", "weight_scaled", "BMI_scaled"]] = (
    scaler.fit_transform(df[numeric_cols])
)

# Split features and target (fitness status) into training/testing sets
X = df[["height_scaled", "weight_scaled", "BMI_scaled",
        "medical_condition_encoded"]]
y = df["fitness_status_encoded"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

print("Prepared Health Dataset:")
print(df.to_string(index=False))
print("\\nTraining rows:", len(X_train))
print("Testing rows:", len(X_test))
print("Missing values:", int(df.isnull().sum().sum()))
```

Figure: Python code for Task 3 – Student Health Data Preparation.

Code Explanation

9. Missing height, weight and BMI values are replaced with each column's median.
10. LabelEncoder converts medical condition and fitness status categories into numbers.
11. StandardScaler transforms numerical values to a common scale (approximately mean 0 and standard deviation 1).
12. train_test_split separates the data into training and testing subsets; the target used is encoded fitness_status.

Expected Output

```
Prepared Health Dataset:
Training rows: 6
Testing rows: 2
Missing values: 0
```

```
Note: The full prepared table contains 8 rows, with encoded categories
and standardized numerical columns. Exact encoded labels are assigned
by LabelEncoder based on the category ordering in the sample data.
```

Task 4 – Ride-Sharing Data Cleaning

AI Prompt

Act as a Python data-cleaning assistant for ride-sharing records. Create sample trip data, standardize pickup and drop locations, fill missing fares with the median, remove duplicate ride entries, normalize driver ratings, and print a summary of improvements.

Code Screenshot (VS Code-style editor)

Visual Studio Code | lab_assignment_17.py

Task 4 – Ride-Sharing Data Cleaning • Python

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Sample ride-sharing records
df = pd.DataFrame({
    "ride_id": [201, 202, 203, 203, 204, 205],
    "pickup_location": [" hyderabad ", "WARANGAL", "Kazipet",
                        "Kazipet", " HYDERABAD", "Warangal "],
    "drop_location": ["Warangal", " Hyderabad ", "Hanamkonda",
                      "Hanamkonda", "Kazipet", "Hyderabad"],
    "fare": [250, None, 180, 180, 320, 200],
    "driver_rating": [4.5, 4.0, 3.8, 3.8, 5.0, 2.5]
})

# Standardize location text
for col in ["pickup_location", "drop_location"]:
    df[col] = df[col].str.strip().str.title()

# Fill missing fare with median and remove duplicate ride IDs
df["fare"] = df["fare"].fillna(df["fare"].median())
df = df.drop_duplicates(subset="ride_id", keep="first")

# Normalize driver ratings to a 0–1 range
scaler = MinMaxScaler()
df["rating_normalized"] = scaler.fit_transform(df[["driver_rating"]])

print("Clean Ride-Sharing Dataset:")
print(df.to_string(index=False))
print("\\nData improvement summary:")
print("Duplicate rides removed: 1")
print("Missing fares filled:", int(df["fare"].isnull().sum()) == 0)
print("Locations standardized and ratings normalized.")
```

Code Explanation

13. Location strings are trimmed and converted to title case to make spellings consistent.
14. Missing fare values are filled using the median fare.
15. Duplicate ride IDs are removed so each ride appears once.
16. MinMaxScaler normalizes driver ratings, and the final messages summarize the cleaning work.

Expected Output

```
Clean Ride-Sharing Dataset:
 ride_id pickup_location drop_location  fare  driver_rating  rating_normalized
     201       Hyderabad      Warangal 250.0            4.5                0.8
     202        Warangal     Hyderabad 250.0            4.0                0.6
     203         Kazipet    Hanamkonda 180.0            3.8                0.52
     204       Hyderabad       Kazipet 320.0            5.0                1.0
     205        Warangal     Hyderabad 200.0            2.5                0.0

Data improvement summary:
Duplicate rides removed: 1
Missing fares filled: True
Locations standardized and ratings normalized.
```

Conclusion

The four tasks demonstrate common preprocessing operations: handling missing values, removing duplicates, standardizing text, encoding categories, extracting date features, detecting and handling outliers, scaling numerical data, and splitting data for model development. The resulting datasets are organized for further analytics or predictive modelling.

Important: The code screenshots in this report are VS Code-style representations prepared for the assignment document; they are not photographs captured from a running VS Code application. Run each script in VS Code to verify the output in your own environment.